Ofrenda — Product Requirements Document

Version: 2.0 · **Date:** 2026-10-28 18:03 IST **Status:** Awaiting build approval. Design settled. **Deadline:** 2026-11-01 12:29 IST (2026-10-31 23:59 PDT) — **~90 hours remaining Supersedes:** `PREREGISTRATION.md` (mine, 2026-10-27) and Collaborator B's `FREEZE SPEC` (2026-10-27). Both are retained as the historical record and must not be edited further. Where this document and either prior spec disagree, **this document governs.**

---

1. Overview

Ofrenda is a contest entry for Hacktoberfest Week 4, "Ghost in the Machine." It takes an abandoned open-source project and does two things with it:

1. **Forecasts when it stopped answering people**, not when it stopped committing — producing two nested probability curves whose gap is the "ghosting band."
2. **Interviews it.** A local open-weight model speaks as the project, citing its own issues and its deprecation notice, and says "the archive is silent on that" when it has nothing.

The corpse is `request/request`: the deprecated Node.js HTTP client. Its death is pinned to three dated events (O-11 settled against primary sources — logged §20): maintenance mode declared **2019-03-30** in issue #3142, which is **still open** (437 comments, ~3k +1s) and promises "*releases will be fully automated, any merge into master will be published*" — so post-2019 releases are bot-authored, and the death announcement is itself an unanswered thread the ghost can cite. npm deprecation **2020-02-11** (registry `time` field, v2.88.2). README rewritten as fully deprecated ~June 2020 (to be pinned via commit history at probe time). The repo still received pushes in 2024 and was never archived. **Expected-shape anchor: 2019-03-30.** It is held out of training, so its forecast is entirely out-of-sample.

Nothing has been built. No data has been pulled. The probe has not run.

---

2. Problem and thesis

Thesis. A project dies years before its last commit. Maintainers stop answering strangers long before they stop pushing code, and that gap is the difference between *abandoned* and *dead*.

The last-commit date is the common proxy for "is this maintained?" and it arrives late. Ofrenda tests whether a social signal — a maintainer answering an outsider — gives earlier warning than code activity does.

Why it is testable. There is no "death date" field in any dataset, and death is a judgement, not an observation. So the project predicts something observable instead: *whether a maintainer answered an issue*

in the 18 months after a reference date T.

The definition that matters. A comment counts only when `author_association` is `OWNER`, `MEMBER` or `COLLABORATOR`, the author is not a bot, and the commenter is not the person who opened that issue. Reviewing a contributor's pull request is not answering a stranger. That distinction is the thesis, and it must survive into the code.

The label is a proxy for social death, not death. The write-up must say this in those words.

---

3. Goals and non-goals

Goals

- Submit a complete, rule-compliant entry before the deadline.
- Test one claim with a pre-registered analysis: social signals are **not materially worse** than code signals at predicting maintainer silence.
- Produce calibrated probabilities with intervals, never bare point estimates.
- Ship an interview with `request/request` that is grounded in its own record.
- Enter **all 19 available prize categories** (all except Arduino) with genuinely meaningful use.
- Publish a write-up that states its limitations before a judge finds them.

Non-goals

- Reviving the project. Revival is an epilogue: a proposed PR, never claimed to work.
- Predicting a death date. TabPFN has nothing to condition on at n = 1.
- Naming or puppeting a maintainer. The ghost speaks as the project, never as a person.
- Winning more than one category. The contest allows one win per challenge.
- Critiquing the people who burned out maintaining `request`. Register: elegy, not autopsy.
- Best-in-class statistical novelty. Two arms and one honest test beats four arms and no time.

---

4. Success criteria

#	Criterion	Measure	Status
1	**Primary test**	Non-inferiority: one-sided CI upper bound on `AUC_A − AUC_B` below 0.07, paired DeLong + repo-clustered bootstrap	**Corrected in this version**
2	Calibration	Brier Skill Score vs base rate + reliability diagram	Settled
3	External validity	Held-out archived repos at `T = archive − 36mo`	Settled

#	Criterion	Measure	Status
4	Probe passes	Coverage table, label mix, row counts, calls/repo all printed	Settled
5	Interview quality	25 scored questions: entailment + abstention. Zero failures in 25 bounds failure at ~12% (rule of three)	Settled
6	Compliance	Repo starts at freeze commit; demo recorded; post in English; tags correct	Checklist §18
7	Coverage	19 of 19 categories entered, each with a one-sentence meaningful use	§7

The primary test, stated correctly

This is the most important correction in this document, and it changes the conclusion you are allowed to draw.

The thesis predicts **parity**: if social signals lead, then at a fixed T the social arm should predict the next 18 months *at least as well* as the code arm, because the code signal has not moved yet. A superiority test would report that prediction as a failure.

```
H0: AUC_A − AUC_B ≥ 0.07     (social is materially worse)
H1: AUC_A − AUC_B < 0.07     (social is not materially worse)
One-sided α = 0.05
Non-inferiority declared if the upper bound of the one-sided CI
on (AUC_A − AUC_B) falls below 0.07.
```

ΔAUC 0.07 is the non-inferiority margin, not a success threshold. Superiority is reported descriptively and never used as the pass/fail criterion.

If the result is inconclusive — CI straddling the margin — that is reported as inconclusive. It is not reframed as a win.

---

5. Scope

Two artifacts, one corpse.

Forecast. Two nested probabilities per reference date across a nine-point grid, a ghosting band with a bootstrap ribbon, and a lead-time analysis. Descriptive, not decorative.

Interview. A local Gemma model, retrieval over the corpse's cached archive, strict citation-or-silence, plus an altar page (final commit, oldest unanswered issue, the stale line in the README, the forecast curve).

19 integrations, six load-bearing and thirteen thin but real. See §7.

Out of scope: revival beyond a proposed diff; any UI beyond the altar page; any model training beyond TabPFN's in-context fitting and one Tinker fine-tune.

6. The 19 categories

All except Arduino, which requires physical hardware. The contest bar is "*does the project use that technology in a meaningful way?*" — not "*would the project be impossible without it.*" Each entry below must do one real thing in the product and be describable in one sentence.

Featured ($200 each)

Category	Meaningful use in Ofrenda
TabPFN	The forecast itself. Two in-context classifiers over ~300 repos — exactly TabPFN's small-n tabular regime
Gemma	The interview runtime. Runs locally via Ollama; the ghost is Gemma
Tinker	Fine-tune on the project's own commit messages and changelog so it speaks in the project's cadence; baseline is vanilla Gemma
Render	Hosts the altar page and the interview endpoint
DigitalOcean	GPU Droplet serving `gemma3:12b` for visitors without a GPU — the "someone else's laptop" path

Partner ($100 each)

Category	Meaningful use in Ofrenda
MongoDB Atlas	Vector store of issues and commits for retrieval; Atlas Vector Search does the semantic leg
Tiger Data	pgvector hybrid search — exact symbol names need BM25, concepts need vectors
Temporal	Durable ingestion workflow. A 4-hour pull that must survive failure and resume
Sentry	Agent tracing — tokens, latency and cost per stage. The debugging narrative is the submission
ElevenLabs	The ghost speaks its own changelog aloud; also transcribes the project's conference talk for indexing
SerpApi	Finds forks, successor projects, "is this dead?" threads and the deprecation announcement itself
Browserbase	Reads the dead docs site and archive.org snapshots — pages with no API
Auth0	Consent gate: a human must approve before the ghost is published or speaks publicly. Token Vault holds the repo token
Backboard	Cross-session memory — the ghost remembers what you asked last time
Ultramarine	Long-document reasoning over ten years of issues: "what was it trying to become?" Cost of each run reported
Mastra	Agent orchestration for the interview loop (TypeScript sidecar against the Python core)
GitHub Copilot	Copilot CLI / coding agent produces the modernization PR that is the revival epilogue
Entire	Agent session record in the write-up. **Product unverified — confirm what it does before claiming it**
.tech domain	The project lives at a `.tech` address, linked in the write-up

Rule: no decorative calls. If an integration is removed and the product behaves identically, it does not belong in the submission. Every one of the 19 must appear in the honest-ledger section with what it actually bought.

---

7. Data and corpus

Parameter	Value
Target n	300 repositories (150 if the probe kill-switch fires)
Dead cohort	`pushed:<2021-01-01 stars:>100 archived:false`
Archived stratum	≥ 60 of 300 → 40 train, 20 held out, blocked by language
Alive controls	`pushed:>2025-06-01`, matched on star and age band
Age floor	Created after 2012; earlier issue history is unreliable or migrated
Languages	JavaScript and Python only
Excluded	`request/request` — specimen, never training data
Auth	`gh auth token` — verified 5,000 core/hr, 30 search/min

Endpoints and cost

One endpoint cannot produce a valid label. `/issues/comments` also returns comments on pull requests, so without a join the label degrades to "responded to anything" — which is not the thesis.

- `/issues?state=all&since=` → issue author, `created_at`, `pull_request` filter
- `/issues/comments?since=` → comment author, `author_association`, timestamp

≈ 18,000 calls for 300 repos ≈ 3.5–4 hours at 5,000/hr. REST, not GraphQL: GraphQL is billed by node cost and blows the budget on heavily commented corpses.

Fetch in stages — but every stream is required. An earlier draft said "comments first — labels need only these." That is not implementable, and is corrected here (logged §20): `y1` requires the issues join (PRs are identified by the `pull_request` key on issue objects, and each issue's opener comes from the same fetch), and `y2` must also see commits and releases. No label can be computed without all four streams. Staging therefore means: fetch repo meta first and bail on degenerate or pathological corpses before paying for the remaining streams — not comments-only labels.

Store events, not features. Raw NDJSON at fetch time; labels and features are pure functions over that cache. Relabelling is then free and competing label definitions can be tested against one fetch. Aggregates bake in window choices and are expensive to retract.

The probe is rate-bound, not latency-bound

18,000 calls at 5,000/hr is 3.6 hours regardless of per-repo latency, provided concurrency saturates the budget. **So the probe logs calls per repo, not wall-clock per repo.** At 60 calls/repo the full pull is 3.6h; at 120 it is 7.2h and the timeline breaks. That is the number that predicts the schedule.

---

8. Labels and masks

At reference date `T`, outcome window `[T, T + 18 months)`.

Target	Equals 1 when	Reported as
`y1`	No maintainer answered a stranger's issue in the window	`P(≥ GHOSTING)`
`y2`	No human activity of any kind in the window	`P(≥ DEAD)`

Nesting holds by construction: `y2 = 1 ⇒ y1 = 1`. The three states read off them are **ALIVE** (`y1 = 0`), **GHOSTING** (`y1 = 1, y2 = 0`), **DEAD** (`y2 = 1`).

Monotonicity. Fit `y2` conditionally on `y1 = 1` and multiply: `p2 = p1 · P(y2 | y1 = 1)`, so `p2 ≤ p1` always. Isotonic projection is a no-op check, not a correction. Report the violation rate either way.

Masks

1. `T ≥ creation + 12 months` — features need history.
2. **`≥ 1 human event in [T − 12 months, T)`** — without this, a repo silent since 2016 yields a trivially DEAD row for every later `T`. That is free accuracy and a meaningless AUC. This is the single easiest way to produce a beautiful, wrong result.

Print post-mask row counts and `y2` prevalence before trusting the second curve. Mask 2 removes the rows that supply most `y2 = 1` cases.

---

9. Features and arms

Banned class

Any recency transform over an event stream the outcome window also reads. The clearest case is `days_since_last_human_commit` — nearly the label in disguise, which would make the thesis unfalsifiable by proving the trivial "recent ⇒ alive."

`docs_vs_code_divergence` is dropped: its denominator is days since the last code change, which is exactly such a transform.

Registered features

ID	Arm	Feature
`f1`	A	Release-cadence decay: ratio of inter-release interval medians, recent 24mo vs prior 24mo. **Requires a `never_released` indicator** — many dead repos never released at all
`f2`	A	Commit-interval Shannon entropy, 12mo
`f3`	B	Open-issue backlog slope, 12mo. **Human-opened only** — dependabot corrupts it
`f4`	B	Maintainer response-latency trend: median recent vs prior 12mo, answered stranger issues
`f5`	B	Response-rate decline: share of stranger-opened issues answered, recent vs prior
`f6`	B	Maintainer-comment volume ratio, recent vs prior 12mo
`bf50`	control	Authors covering 50% of commits in `[T−12mo, T)`

The arms must be capacity-matched

A naive split gives Arm A two features and Arm B four. A non-inferiority test where one arm has twice the capacity is confounded **in the direction of the hypothesis** — B has more chances to capture signal, so "parity" is partly capacity.

Primary test uses two features per arm: - $\mathbf{A_2}$ = { `f1`, `f2` } - $\mathbf{B_2}$ = { `f4`, `f5` } — the two most thesis-relevant social signals

`f3` and `f6` are reported descriptively only. No new features are invented to pad Arm A.

Every fit also carries background controls: `log10(stars)`, repo age at T, language, created-year cohort. These are matched-pair controls, not competitors to the signal under test.

Say this in the write-up

With the strongest code feature banned, feature counts differ even after matching. Report the counts beside the result, and state that part of any gap is structural.

---

10. Analysis plan

Nothing here may change after the first outcome is observed. Additions are permitted only if timestamped before the probe completes and logged in the change table (§20).

Split

Leave-one-repo-out over `(repo, T)` rows. Several reference dates per repo would otherwise leak a project's own history across the split. TabPFN is in-context, so held-out repos are simply excluded from the context set.

Comparison hierarchy — fixed-sequence gatekeeping

Not a pairwise matrix. Six tests at α = 0.05 gives ~26% family-wise error.

Order	Test	Rule
1. Primary, confirmatory	**A_2 vs B_2 non-inferiority**	One-sided α = 0.05, margin 0.07
2. Descriptive	Superiority A_2 vs B_2, full-feature A vs B	No inference claimed
3. Descriptive	Per-bus-factor-quartile AUC	Interaction only, no inference

B′ and C arms are **dropped** to fit the timeline. Consequence: the claim "*the social signal survives structural controls*" comes out of the write-up entirely. Registering it and not testing it would be worse.

Instruments

- **Paired DeLong** for the AUC difference — the models share rows, so pairing is far more sensitive than comparing point estimates. An unpaired SE at n = 300 is ~0.045, which is why a bare 0.07 threshold proves nothing.
- **Repo-clustered bootstrap** (≥1,000 resamples) as the confirmatory check. DeLong assumes independent rows; rows from one repo are not independent. If the two disagree, the bootstrap governs.
- **Brier Skill Score** vs base rate (`1 − Brier/Brier_ref`). Raw Brier is uninterpretable.
- **Reliability diagram** — Brier conflates calibration with resolution.
- **Band ribbon** from the same bootstrap. The band is a difference of two probabilities, so calibration error compounds in the subtraction.

Lead-time analysis (descriptive, cheap, high value)

For each repo, find the first T at which each arm's `p(DEAD)` crosses 0.5, and compare that date to the actual death date. Non-inferiority shows social is *as good*; only lead time shows it is *earlier* — which is what the thesis actually claims. Costs no α and produces the chart the argument needs.

---

11. External validation

`archived:true` is GitHub's own death certificate and the only ground truth here that is not an opinion. It is also selected — owners archive what they care about — so it is a stratum, never a clean sample. `request/request` is permanently outside this validation set.

Test	Reference date	Purpose
Primary	`T = archive − 36 months`	Window closes 18 months before declaration; real uncertainty remains
Sanity	`T = archive − 18 months`	Window ends at declaration — near-tautological, boundary check only

20 of the 60 archived corpses are held out entirely. Validating on repos seen in training is in-sample and worth nothing. **Archival status is a label source and must never enter the feature schema.**

---

12. The interview

- Local Gemma via Ollama. `gemma3:12b` recommended (~8 GB); `2b` will not produce a convincing interview.
- Commits act as memory, issues as arguments, docs as self-image.
- Every answer carries a citation (issue #, commit SHA, README line) or the sentence **"the archive is silent on that."** No improvisation.
- **Two voices, split by artifact.** Elegy in the interview display. Plain autopsy-descriptive prose in evidence and methodology sections. Mixing them makes either the story or the method unreadable.
- The ghost speaks as the project. No individual maintainer is named, and no claim about anyone's state of mind goes in unsourced.
- **The interview is n = 1.** It is a story, not evidence, and the write-up must not present it as evidence.

---

13. Probe — 20 repos, before the full pull

The probe exists to break the pipeline, not confirm it. A random sample tests the happy path; spend the 20 on cases that would kill the full pull.

Group	n	What it breaks
Issues disabled	3	`f4` undefined; how large is the hole?
Zero releases ever	3	`f1` divide-by-zero path
Bot "maintainer"	3	Bot filter correctness
Single commit	3	Degenerate history
`archived:true`	3	Label-source join correctness
Known-alive controls	5	If all 20 return DEAD, the label is broken, not the corpus

Print: per-feature coverage table · label distribution · post-mask row counts and `y2` prevalence · monotonicity violation rate · **calls per repo**.

Freeze labels before printing any label. No threshold tuning after the first look.

Kill-switch: > 90 calls/repo ⇒ 300 repos exceeds 5.4 hours ⇒ descope. Ladder: 300 → 150 → 100 → single cohort. Decided by the probe, not by optimism.

---

14. Risks

Risk	Effect	Mitigation
Trivially DEAD rows	Inflated AUC	Mask 2 (§8)
Mask removes too many DEAD rows	`y2` too few positives	Print post-mask counts in the probe
Maintainer rule biased toward ALIVE	Result drifts to null	`author_association`; top-5 only as robustness with $K \in \{1,3,5\}$
Recency features	Thesis unfalsifiable	Ban the whole class
Capacity mismatch between arms	Part of the gap is built in	Matched 2-vs-2 primary
Superiority misread as the test	**A correct result reported as failure**	Non-inferiority (§4)
API budget	Pull exceeds the window	Calls/repo kill-switch
19 integrations eat the schedule	No write-up	Timebox each; writing starts Day 1
Compliance miss	Disqualification	§18 checklist
Two AIs agreeing	Shared blind spots look like consensus	Treat the probe, not the agreement, as the first real test

---

15. Limitations — to publish, verbatim

1. Two definitions of DEAD coexist: declared (archived) and inferred (silence). The model learns a blend.
2. The archived stratum is selected. Owners archive what they care about; `request` was deprecated but never archived.
3. Missingness correlates with age. Pre-2012 repos have unreliable issue history.
4. Issue features are undefined where issues are disabled, and not at random.
5. n = 300 is small for three latent states. Report intervals, never bare point estimates.
6. The label is a proxy for social death, not death itself.
7. The interview half is n = 1: a story, not evidence.
8. The primary claim is non-inferiority, not superiority. Parity is the prediction.

---

16. Timeline — ~90 hours

Assumes the probe runs tonight. Every day of delay comes straight out of the write-up.

Window	Work	Hours
Oct 28 evening	Claim credits; sign TabPFN licence; start `gemma3:12b` pull in background; run probe; lock n	5
Oct 29	Full pull; derive; features; two fits; band + lead-time charts; **write-up skeleton**	13
Oct 30	Interview spine (index → ask → cite); altar page; first 10 integrations	13
Oct 31	Remaining 9 integrations; deploy; demo video; write-up draft	13
Nov 1 until 12:29	Final write-up; compliance checklist; publish	5
	Total	**49**

~90 hours available, ~49 committed. The slack is for integrations that fight back — and several will. Writing is drafted from Day 1 because it carries the heaviest weight.

---

17. Open decisions

ID	Decision	Recommendation
O-1	Project name: "Ofrenda" or "Ghost repo interview"	Ofrenda
O-2	Corpse = `request/request`	Lock
O-3	Pull `gemma3:12b` (~8 GB)	Yes — 2b cannot carry an interview
O-4	**Build approval**	**The only blocker**
O-5	Credits to claim: Tinker, Ultramarine $5, .tech	Claim all three; ~10 minutes
O-6	Language strata JS + Python	Lock
O-7	Demo recorder: vhs vs asciinema	vhs
O-8	Maintainer-mortality feature	**Dropped.** Unregistered, +2 calls/repo, and inapplicable to an org-owned corpse
O-9	"Survives structural controls" claim	**Struck** — B′ and C are cut
O-10	Primary claim	**Settled: non-inferiority** (§4)
O-11	`request` deprecation date	**Settled 2026-10-28** on primary sources: maintenance mode **2019-03-30** (issue #3142, still open), npm deprecation **2020-02-11** (registry `time`, v2.88.2), README ~**Jun 2020**; expected-shape anchor 2019-03-30 (§1, §20)

---

18. Compliance checklist

Verified against the challenge page and official rules on 2026-10-28.

Before building

- [ ] DEV account exists and is in good standing (entrants must be DEV members)
- [ ] Submission repo created after 26 Oct, 9:00 AM PDT (21:30 IST); reuses no older code
- [ ] `marigold/` stub removed from the workspace — **done, 2026-10-28**
- [ ] TabPFN non-commercial licence signed at platform.priorlabs.ai
- [ ] Promo credits claimed: Tinker, Ultramarine $5, `.tech` domain
- [ ] Project name decided

In the repo

- [ ] README lists every commit made after the deadline
- [ ] README credits all prior work and open-source code used
- [ ] Repo link opens without a login

In the post

- [ ] English
- [ ] Tags `devchallenge`, `hf26challenge`
- [ ] Sections: What I Built · Demo · Code · How I Built It · Why Does Open Innovation Matter? · Prize Categories
- [ ] Demo is a deployed link or a video walkthrough (plan: real video, not a GIF)
- [ ] Names the open-source AI used, how the project is built around it, and why open innovation matters — with one concrete comparison against a closed option
- [ ] Lists all 19 categories entered, each with its meaningful use
- [ ] Optional: DevRelay session; "The Spirit of It"
- [ ] `request/request` facts checked against sources; no unsourced claims about anyone's state of mind
- [ ] Limitations section included

Publishing

- [ ] Post stays a draft until final — **only the first entry submitted is judged**
- [ ] Published well before 2026-11-01 12:29 IST. The sponsor's clock is the official clock

---

19. What changed from the two prior specs

#	Change	Why
1	Primary test: superiority → **non-inferiority**	Parity is what the thesis predicts; a superiority test reports a correct result as failure
2	Arms capacity-matched to 2 vs 2	2-vs-4 confounds the test toward the hypothesis
3	`f3` moved from Arm A to Arm B	Backlog slope is issue-derived; in A it blurs code-vs-social
4	Row mask 2 added	Trivially DEAD rows inflate AUC
5	Maintainers via `author_association`	Top-5 authors biases the label toward ALIVE
6	`docs_vs_code_divergence` dropped	Fails the recency ban on its own denominator
7	`B′` and `C` arms dropped	Timeline. "Survives structural controls" struck with them
8	Kill-switch: wall-clock → **calls per repo**	The pull is rate-bound, not latency-bound
9	19 categories restored	Contest bar is "meaningful use," not "irreplaceable" — the stricter test was ours, not theirs
10	`0–8` maintainer-mortality dropped	Unregistered, costly, inapplicable to an org-owned corpse
11	This document supersedes both prior specs	Three sources of truth undermine the pre-registration claim
12	Lead-time analysis added	Non-inferiority shows "as good"; only lead time shows "earlier"

20. Change log

Date	Change	Author
2026-10-28 18:03	v2.0 — consolidated, 19 categories restored, non-inferiority adopted, arms matched, statistics reduced to fit the timeline	—
2026-10-28 18:32	**§1/§7 corrections.** Corpse dates pinned: maintenance mode 2019-03-30 (issue #3142, still open), npm deprecation 2020-02-11 (registry `time`), README ~Jun 2020 (pending commit-history pin); expected-shape anchor = 2019-03-30. "Comments first" staging replaced: labels require all four streams (issues join for PR numberset + openers; commits + releases for y2)	AI-A
2026-10-28 18:32	**Two derive defects fixed pre-hoc (nothing had run).** (1) `human_activity()` skipped commits with no GitHub-linked login → false-DEAD rows. Fix: `is_bot_commit()` checks GitHub login first, then git author/committer name+email (`[bot]` substring or denylist). (2) Comments by deleted/ghost accounts (`user: null`) were invisible → now count as human in activity (unknowable identity ≠ bot)	AI-A
2026-10-28 18:32	**Pagination caps pre-registered:** issues 40 / comments 60 / commits 40 / releases 5 pages × 100. Issues & comments fetched **newest-first** so caps truncate at the old end — capped repos lose early-T rows via mask 2, never late labels. `truncated` flags kept and reported	AI-A
2026-10-28	**Probe pre-registration (before any label prints):** T grid = Jan 1 of 2015…2023 (9 points, annual); all labels frozen before printing — no threshold tuning after the first look; print label mix at T = 2023-01-01,	AI-A

Date	Change	Author
18:32	post-mask row counts, y2 prevalence, monotonicity violations, ghost-user counts, calls/repo	
2026-10-28 18:32	**Probe roster = §13's 20 + the corpse.** `request/request` is fetched and derived during the probe (excluded from every fit regardless — specimen rule intact): front-loads its ingest and catches specimen-specific truncation before the full pull	AI-A
2026-10-28 18:32	**Kill-switch operationalized:** median core calls/repo across the roster > 90 ⇒ descope the corpus 300 → 150 (ladder per §13: 300 → 150 → 100 → single cohort). Decided by probe output, not optimism	AI-A
2026-10-28 19:12	**Probe run 1 aborted at the 40-min wall — fetch-layer defects, nothing derived, no label printed.** Evidence from run 1's log/cache: (1) three controls died on `IncompleteRead`, a transport error that `gh.get` did not retry; (2) `axios` cached EMPTY issues/comments/commits after silent 403-exhaustion returning `None` — silence written to disk as if true; (3) search-endpoint rate headers leaked into the core limiter, stalling every search 53 s. Fixes: search/core limiter isolated; `IncompleteRead` / `RemoteDisconnected` /connection errors retryable; 403 honors `Retry-After` and `X-RateLimit-Reset`; pagination raises if a stream breaks after page 1; empty-issues guard when `has_issues=true`; 0.5 s/page pacing. Run-1 log kept as evidence (`probe_out/probe_2026-10-28.log`); partial cache discarded; run 2 fetches fresh	AI-A
2026-10-28 19:20	**Backend registrations (pre-fit, while run 2 fetches).** Feature arithmetic pinned: `f1` = median inter-release interval ratio over [T−24mo,T) vs [T−48mo,T−24mo), with a `never_released` companion indicator (zero releases anywhere in the record); `f2` = Shannon entropy (bits) of intervals between **human** commits in [T−12mo,T), binned in days (<1 / 1–3 / 3–14 / 14–45 / 45–120 / ≥120); `f4` = median issue→first-maintainer-response latency ratio (recent vs prior 12mo across [T−24mo,T)), first-response **after T is banned** (look-ahead rule); `f5` = recent_share − prior_share of stranger issues answered (differences, not ratios, so no division holes); `f6` = maintainer-comment volume ratio (divisor 0 → None) with both counts emitted descriptively; `f3` = OLS slope of monthly human-opened unresolved non-PR backlog (descriptive only); `bf50` = top human commit authors covering 50% of [T−12mo,T) commits. **Global ratio convention: denominator 0 → None, never imputed.** Rows land in `derived/rows.ndjson`. Annual grid constant moved to derive as `ANNUAL_GRID` (same math, one definition; masked rows via `derive.rows_for`)	AI-A
2026-10-28 19:24	**Corpus-pull design locked before run 2 lands** (implements §7). Cells = language × star-band × created-band: star bands 100–300 / 300–1 000 / 1 000–3 000 / ≥3 000; created bands 2013–2015 / 2016–2018 / 2019–2020 (age floor: nothing created before 2013); only Python + JavaScript. **Strata shares fixed at 0.6 dead / 0.2 archived / 0.2 alive**, language split 50/50 within strata; alive controls are matched on stars + age band by giving each cell an alive quota proportional to the dead cohort's actual cell counts (largest-remainder rounding). Dead query = `pushed:<2021-01-01 archived:false fork:false <bands>`; archived = same with `archived:true`; alive = `pushed:>2025-06-01 archived:false fork:false <bands>`. Candidates taken in search-result order, **page 1 only** (per_page=100); a short cell's quota redistributes over cells that still have page-1 candidates, fixed order. Roster-vet rejects stale-index contradictions (fork flag, star/created band mismatch, dead candidates still pushing, pre-2013 creation). `pull.py` CLI: `--n` (default 300), `--discover-only`, `--fresh`, `--limit`; roster lives at `pull_out/roster.json` (chosen candidates + per-cell receipts embedded); a `--n` mismatch rebuilds the roster. Pull is serial and journaled (`pull_out/journal.ndjson`, one row per attempt); journaled-ok repos are skipped for free on restart, failed entries are retried; **5 consecutive failures abort** the run.	Pull order: dead → archived → alive, cells in fixed order

Date	Change	Author
2026-10-28 22:42	**Search-throttle degradation policy (second launch attempt).** The first pull launch died 10 min into roster discovery: a search query exhausted gh.get's 8-retry 403 budget and the new `RateLimited` hardening (correct for core streams) crashed the run at the search layer — GitHub's search endpoint applies its own secondary abuse detection on top of the 30/min cap, and ~10 min of 27/min paging trips it. Policy: `_search_page` now owns search retries — on a 403/429 episode it waits 60 s (lets the per-minute window roll) and re-queries, up to 3 rounds; only then does the cell go **unfilled** (recorded in the roster receipts, redistributed, never silently dropped). Search pacing raised 2.2 → 3.0 s (20/min) for margin. Discovery is idempotent and restartable by design; unfilled deficits are visible in `pull_out/roster.json`, never cached as truth. Pull logs are unbuffered (`python -u`) from here on — the buffered tee hid all diagnostics for 10 minutes	AI-A
2026-10-28 21:22	**Probe run 2 verdict — kill-switch FIRES: n = 150 locked.** Run 2 (log `probe_out/probe_2026-10-28_run2.log`, summary `probe_out/probe_summary.json`): 4 of 21 roster repos fetched successfully (psf/requests 142 calls, pallets/flask 142, request/request 120, twjiang/fact_triple_extraction 5); all four cached real data under the registered caps — no silence cached. **Median calls/repo = 131 > 90 ⇒ DESCOPE 300 → 150** (pre-registered §13 rule applied as written; honest caveat: computed on n=4 successful fetches because 17 of 21 failed — sample dominated by the two fat controls, which is exactly the rate-bound case the switch exists to catch). Ladder stops at 150; strata rescale to 90/30/30 per §20 19:24. **Invariants on the fetched sample: monotonicity violations 0; both fetched controls have ALIVE rows; the corpse reads GHOSTING at T=2023-01-01 (y1=1, y2=0 — no maintainer answered in 18 months while humans still commit) — the thesis anchor is live.** Failure-mode findings (drive fetch hardening, this row): (1) sustained secondary-limit 403s exhausted gh.get's 4-retry budget and returned None, which fetch_repo misreported as "repo not found" for repos that exist (axios, fastapi, 15 others); (2) two transport breaks mid-pagination (express page 27, reddit-archive page 3) hard-aborted those repos. Hardening round 2, effective for the full pull: gh.get raises `RateLimited` instead of returning None on exhausted 403/429 retries (None now means genuine 404/422), retries 4→8, 403 backoff honors Retry-After up to 130 s, page pacing 0.5→0.75 s, pull adds a 2 s pause between repos. Failed probe repos re-fetch on the pull (per-stream cache makes retries cost only missing streams). No analysis-plan text changed — verdict and fixes only	AI-A

Any change after the probe runs requires a dated row here. No silent edits.